AMATH 482/582: HOME WORK 4

VARUN PANUGANTI

ACMS Department, University of Washington, Seattle, WA
varunp5@uw.edu

Abstract. In this report I trained models that distinguished images of handwritten digits from the famous MNIST data set. I first conducted normalization and PCA Analysis on the data. Then I trained a Ridge Classifier (using cross validation), a Logistic Regression model, and a KNN classifier on different subsets of data. I then presented visualizations on the truncated PCA modes, the reconstructed digits using these PCA modes, and a table comparing the test accuracy of each of the models I trained. I finally concluded my findings and the implications of my report.

1. Introduction and Overview

In this report the problem is to use classification to distinguish images of handwritten digits from the famous MNIST data set. The data set is split into training and test sets. First, the data must be centered and normalized to ensure numerical stability. Then PCA analysis will need to be conducted to preserve the essential information from the original MNIST Data. After that three different classifiers (Ridge Classifier, Logistic Regression, and KNN) will be trained on 3 subsets of data, using cross validation, to obtain the best possible classifier without generalizing on data.

2. Theoretical Background

2.1. **Normalization.** First we want to center our data. We can do this by computing the mean and standard deviation:

$$\mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}$$

and

$$\sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij}-\mu_j)^2}$$

Every feature then becomes:

$$x_{ij} = \frac{x_{ij}-\mu_j}{\sigma_j}$$

Date: March 18, 2026.

2.2. **PCA Analysis.** Once our data is normalized we then want to find the PC modes. Let $U^k = [U_1, ..., U^K]$ which are the top-k left singular vectors. Then we can compute the PCA variance ratio: $1 = \sum_{i=1}^{k} \lambda_i$. We want to determine k, the number of PC modes needed to approximate 85% of squared Frobenius norm.

The percentage of data lost when we construct our approximation can be represented by:

$$R(k) = \frac{\sum_{i=k+1}^{r} \sigma_i^2}{\sum_{i=k}^{r} \sigma_i^2}$$

While this process was carried out using the PCA library, mathematically, we want to find k such that:

$$1 - R(k) \geq 0.85$$

2.3. **Ridge Classification.** After conducting normalization and PCA analysis to obtain our k value, we want to train a Ridge Classifier model.

I used the library RidgeClassifier to train the model. Mathematically the classifier takes the form of:

$$\hat{y} = argmax_{j=1,...,c}(z^T W)_j$$

where W is our set of coefficients that we obtained from training our model, z is the new point we want to classify, and $\hat{y}$ is the class we assign it to due to it having the highest probability.

We can train our model by using Cross Validation which will be explained in the next section.

2.4. **Cross Validation.** Cross validation is used to select a regularization parameter. The regularization parameter allows us to penalize large coefficients of our model, and cross validation on k-folds allows us to generalize our model across different folds of data, both of which prevent overfitting.

The process looks like:

best_validation_error $= \infty$
Chosen $\lambda =?$

for λ in lambdas:
    aux = 0
    for fold in k folds:
        compute:

$$min_{W \in R^{d*c}} ||ZW - Y||_F^2 + \lambda ||W||_F^2$$

        aux $+= \frac{1}{m} \sum_{i=1}^{m} 1_{f(x_i) \neq y_i}$
    average_validation_error $= \dfrac{\text{aux}}{n}$
    if (average_validation_error < best_validation_error):
        best_validation_error = average_validation_error
        Chosen $\lambda = \lambda$

Finally refit:

$$min_{W \in R^{d*c}} ||ZW - Y||_F^2 + \lambda^* ||W||_F^2$$

where λ^* is the chosen value.

2.5. **Logistic Regression.** Alongside a Ridge Classifier model, we will train a Logistic Regression model, which is used for the purpose of binary classification. The model takes the form of:

$$q(x_i; c) = \sigma(c_0 + \sum_{j=1}^{p} c_j x_j)$$

Where q is the predicted probability, x_j is the jth feature, c_j is the coefficient for the jth feature, c_0 is the intercept, σ is the sigmoid function which takes the form of:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The model will be trained by selecting the coefficients which minimize this objective function:

$$\prod_{i \in I} q(x_i; c) \prod_{i \notin I} (1 - q(x_i; c))$$

2.6. **K-Nearest Neighbors (KNN).** K-Nearest Neighbors or KNN, is a supervised non-parametric method, meaning it does not make any assumptions of the functional form of f.

As we already know, the training dataset is stored in pairs $(x_i, y_i)_i^m$, where in this case x_i is the PCA feature vector of the i-th image, y_i is the label, and m is the number of samples.

The label of the new test point is obtained by computing the distance between it and all other data points and taking the k closest points. Then the class label that appears the most frequently is assigned to this new point.

We can determine classification error by using the function:

$$error = \frac{1}{m} \sum_{i=1}^{m} 1_{\hat{y} = y_i}$$

Rather than picking a random number of neighbors we can choose the optimal number of neighbors using cross validation. We can follow the same cross validation process from section 2.4, except test for different k values (where k is the number of neighbors) instead of λ values, and in this case our error function will be the same.

3. Algorithm Implementation and Development

I used StandardScaler from sklearn.preprocessing to scale the X and Y values of my train and test data. I used the library PCA from sklearn to help compute the PCA of my data, and calculate the cumulative energy. I used accuracy_score from sklearn.metrics and cross_val_score from sklearn.model_selection to compute the accuracies of my classifications. I also used sklearn.model_selection to import GridSearchCV which allowed me to carry out cross validation. I used RidgeClassifier and LogisticRegression from sklearn.linear_model to train my ridge classifier and logistic regression models. I used KNeighborsClassifier from sklearn.neighbors to train my KNN classifier [3]. I used matplotlib.pyplot to plot my PC modes and my reconstructed digits [2]. I used Numpy operations like cumsum, to find the the cumulative explained ratio, argmax to find the k value which approximates 85% of the energies, unique to grab all the indices for my final subset, and isin to find the digits corresponding to the subset indices in my select_digits method [1].

4. Computational Results

Figure 1. These are the first 16 truncated PC modes. They are used as the basis for creating the digits in the MNIST dataset. They somewhat resemble the digits, as for example one can see that different curves and lines used to make numbers like 5, 4, and 9 are present in modes like [0, 3] and [2, 1] (coordinates of the grid presented in the figure).

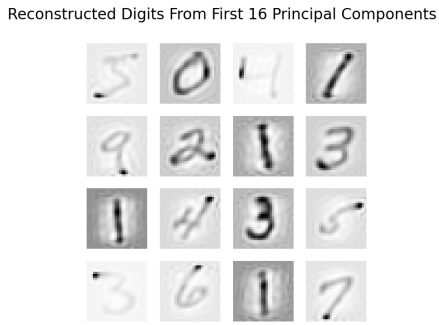

Figure 2. These are 16 approximated digit images reconstructed from the k truncated PC modes. The number of PC modes needed to approximate 85% of energy is 59. While these digits look blurry, they look very similar to the first 16 digits in the original training dataset.

	[1,8]	[3,8]	[2,7]	all
Best Alpha	100	1	100	100
Ridge Classifier Test Accuracy	0.980	0.961	0.979	0.859
Logistic Regression Test Accuracy	0.992	0.980	0.981	0.927
KNN Classifier Best Neighbors Value	N/A	N/A	N/A	3
KNN Classifier Test Accuracy	N/A	N/A	N/A	0.9548

Table 1. When using all digits, the accuracy of both the Ridge Classifier and Logistic Regression model decreased. The most optimal set of digits was [1,8]. When comparing all 3 models, I chose to observe the final column with all digits, as that is the only column that has test accuracy for all 3 models. The model that performed the best out of the 3 was the KNN Classifer, where the optimal amount of neighbors looked for was 3.

5. Summary and Conclusions

In this section I will present my findings. PCA is an effective tool when it comes to preserving essential information from datasets. Figures 1 and 2 show how PC modes that preserve 85% of the data can still recreate the data effectively enough for a viewer to tell what the digit is. Furthermore, based on my findings, I learned that out of the choices of data subsets presented in table 1, and the choice between Ridge Classifier and Logistic Regression models, the Logistic Regression model performs better. When fixing on all indices, and comparing between the Ridge Classifier, Logistic Regression, and KNN Classifier models, the KNN Classifier performs the best.

Acknowledgements

The author is thankful to Prof. Frank for useful discussions about Ridge Classification, Logistic Regression, KNN classification, and Cross Validation.

References

[1] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.

[2] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.